assignment 3

Patterns & Flows: An Everyday Tool

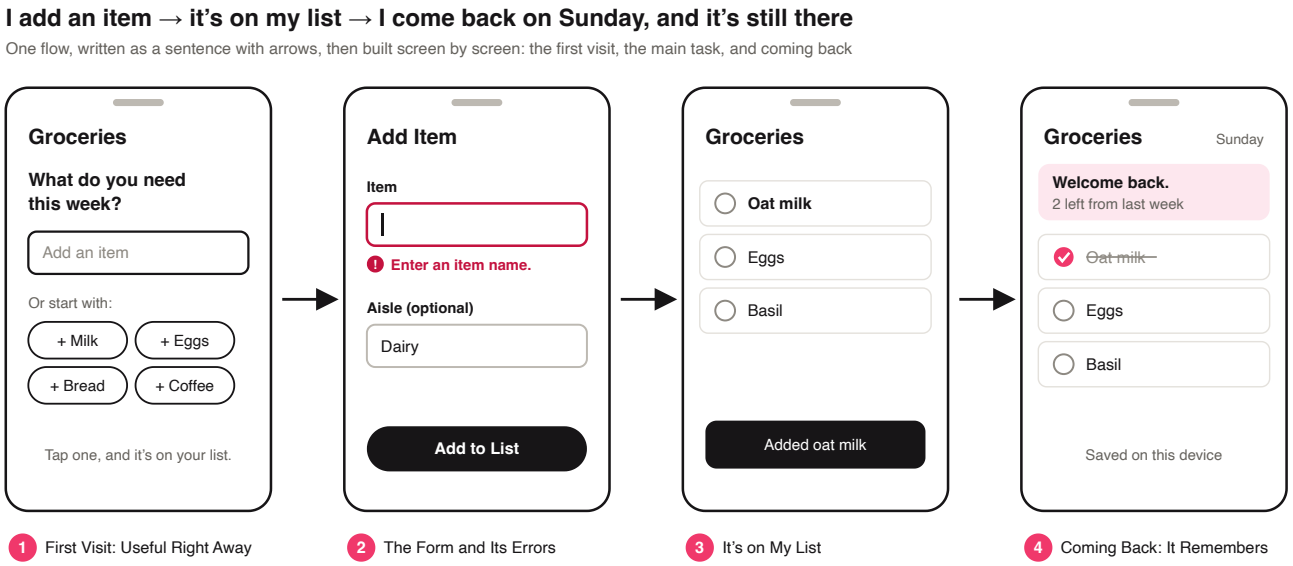
Design, Build, Ship · MPCS 51238 · Autumn 2026

Due Tuesday, October 20, at 5:30 PM

Overview

Software! Last week's site looked real, but it was mostly faked. Now we're going to make something a bit more real: robust user flows, forms with validation, and local data storage. You'll design and build a small tool you'd actually use every day or every week, starting from scratch, around its flows: your first visit, the core task, and reasons to return. It runs in Next.js on Vercel, and it saves to the browser, so it remembers you on your device.

Make every change by directing your agent. Before you build, work out your tool's flows with the agent and study how similar tools welcome someone new. Once it's deployed, have your agent try it as a brand-new user, and fix what it finds.



Steps

1. Pick a Tool You'd Use Every Week

A tool for something you do daily or weekly, started fresh in a new project. Your Assignment 2 design system is a good place to start its style. What problems, big or small, interest you? The more specific and unique, the more we'll have to work with. Some ideas:

- A weekly grocery list
- A bill splitter for dinners
- A check-in for one habit
- A practice log for an instrument
- A chores rotation for roommates
- A log of study sessions
- A job application tracker
- Vocabulary cards
- A packing list for trips you repeat
- A coffee or tea log
- A commute log
- Dishes to try at a favorite restaurant

No sign-in, database, or APIs. Your tool saves to the browser with localStorage, so it remembers someone on the device they used. Accounts and databases start next week.

2. Define Your Flows with the Agent

Start a new Next.js project. In plan mode (`/plan`), describe your tool and work out its user flows with your agent: the first visit, the main task, and coming back. Write the main task as one sentence with arrows: "I add an item → it's on my list → I come back on Sunday, and it's still there." Ask the agent what steps, states, and errors it would add, and choose what to keep.

Then have it draw your flows as a diagram: Mermaid in a `flows.md` file, or any diagram tool you like. Show the diagram in your `README.md`. Build a page for each step, with shared navigation. In Next.js, each page is a route.

This week's recordings show how: Designing Applications with Agents, the lesson on forms, errors, and empty screens, and Starting Assignment 3.

3. Research How Similar Tools Onboard People

Pick three tools that do something like yours. Ask your agent to send subagents to study how each one welcomes someone new: the first screen, what it asks for, and how soon you get something useful. Decide what you'll use in your own first visit.

4. Design the First Visit

Open your tool with nothing saved. The first visit is the onboarding. Good onboarding delivers immediate value: someone new should understand what the tool is for and do something useful right away. Let them learn by doing, not by reading a tour, and don't drop them into an empty screen.

5. Build the Main Task and Its Form

Build the screens. Use shared components: buttons, inputs, and fields with labels and errors, used consistently everywhere.

The form adds to your tool. Its errors sit next to the field and say what to do next: "Enter an item name," not "Invalid input." When there's an error, keep what the person already typed.

6. Give People a Reason to Come Back

What makes you want to return to a product? Decide what brings someone back to your tool daily or weekly, and design around it. Save what you need in local browser storage: lists, progress, etc.

7. Make It Accessible

Everything should work with a keyboard, fields should have real labels, and text contrast should be checked with your agent.

8. Run the First-Run Check

Once your tool is deployed, have your agent try it as a brand-new user. Give it the URL and a goal like "Add three things you need this week." Ask it to go through the first visit and the main task, and to pretend to come back later to use the product. Ask the agent for usability feedback.

Design Requirements

- **Flow:** the initial visit, the core task, and returning work from start to finish.
- **Onboarding:** someone new gets something useful on the first visit, with no dead-end empty screen.
- **Forms:** one column, a label on every field, errors next to the field that say what to do next, and save what the person typed.
- **Coming back:** the app remembers what you saved locally, and there's an experience designed around returning.
- **Components:** use shared components consistently throughout the application.
- **Navigation:** it's clear to users where they are and how to get back to where they were.
- **Accessibility:** everything works with a keyboard, fields have real labels, and text contrast has been checked with your agent.
- **Visual style:** optional, but you are welcome to explore unique visual styles here and apply one.
- **Simplicity:** try to remove unnecessary steps, fields, and interactions.

Tech Stack

Next.js and Tailwind CSS, with your data saved in the browser with localStorage: no database, accounts, or external APIs. Put the code in your own GitHub repository and deploy it on Vercel's free Hobby plan. If the agent suggests a database, a backend, or a package you don't need, push back.

Your project's context file is `AGENTS.md`. If there's a `CLAUDE.md`, move anything useful into `AGENTS.md` and delete it, then check with `/context` that `AGENTS.md` loaded.

Submission

Submit in Google Classroom:

1. **Vercel URL:** a live link to your tool.
2. **GitHub URL:** your repository, with your `README.md` and a commit history that shows your progress.

Assessment

A complete assignment has:

- A live Vercel URL that works
- A new tool built around its flows (the first visit, the main task, and coming back), with a page for each step and shared navigation
- Your flows as a diagram (Mermaid or any diagram tool), shown in your `README.md`
- A first visit that gives someone new something useful right away
- A form whose errors sit next to the field and say what to do next, and that saves what the person typed
- A reason to come back, and a tool that remembers you between visits
- Keyboard access, real labels, and text contrast checked with your agent
- The first-run check in your `README.md`: the agent's usability feedback
- A GitHub repository with commits that show your progress
- You're ready to explain your work in class (below)

A one-shot submission, made by giving this brief to an agent once and turning in the result, is marked incomplete.

This is meant to be an enjoyable design experience. If you're running out of time, focus on the requirements. If you need help, reach out.

Be Ready to Explain

In class, you'll talk through your work. Be ready to explain:

1. **Your design process:** the tool you created, the user you created it for, and what'll bring them back.
2. **Onboarding:** what someone new sees, and how they get something useful right away.
3. **Retention:** what brings someone back, and what your tool remembers.
4. **Your agent's feedback:** what did you learn? What changes did you make?
5. **Agentic workflow:** how did you work with your agent? What was easy, and what was hard?